

KNOW-HOW | LLM-INFERENZ

125 Milliarden Parameter auf zwei Gaming-Karten

Wie die Spezial-Engine Strata ein riesiges Sprachmodell auf Consumer-Hardware bringt – und was der Fork arcfork-strata für Coding-Agenten mit 262 000 Token Kontext daraus gemacht hat

Wer ein großes Sprachmodell lokal betreiben will, greift meist zu llama.cpp oder vLLM. Strata geht einen anderen Weg: eine Engine für genau ein Modell, die Grafikkarte, Prozessor, Arbeitsspeicher und PCIe-Bus gleichzeitig auslastet. Der Fork arcfork-strata trimmt sie auf einen Server mit zwei RTX 3090 und lange Agenten-Sitzungen – und hat die Prompt-Verarbeitung dabei mehr als verdoppelt. Ein Blick unter die Haube, der kein Vorwissen voraussetzt.

EIN Sprachmodell mit 125 Milliarden Parametern gehört eigentlich in ein Rechenzentrum. Selbst stark komprimiert belegen seine Gewichte zwischen 40 und 110 Giabyte – weit mehr, als in den Videospeicher einer Spielegrafikkarte passt. Trotzdem schreibt die freie Inferenz-Engine Strata mit genau so einem Modell auf einem gewöhnlichen PC mit einer 12-GByte-Karte und 64 GByte Arbeitsspeicher 60 bis 95 Wörter-Bruchstücke pro Sekunde – schneller, als ein Mensch lesen kann. Ihr Fork arcfork-strata treibt dasselbe Modell in einer vierbittigen, qualitativ hochwertigeren Fassung auf zwei RTX 3090 mit rund 116 Token pro Sekunde an, liest Quelltext mit über 3000 Token pro Sekunde ein und antwortet auf eine Folgefrage in einem laufenden Gespräch nach 0,17 Sekunden.

Wie das geht, erschließt sich erst, wenn man versteht, was eine solche Engine eigentlich tut und wo bei der Ausführung großer Sprachmodelle die Zeit verloren geht. Dieser Artikel beginnt deshalb bei null: Er erklärt zunächst, wie ein Sprachmodell Text erzeugt und warum dabei nicht die Rechenleistung, sondern das Bewegen von Daten der Engpass ist. Danach stellt er das Modell Qwen3.8-Flash-Next vor, dessen ungewöhnlicher Aufbau die Grundlage für alles Weitere ist, und ordnet die beiden verbreiteten Allzweck-Engines llama.cpp und vLLM ein. Es folgen die Architektur von Strata und schließlich die Arbeit des Forks: was er an der ohnehin starken Basis verändert hat, warum, und was es gebracht hat.

Was eine LLM-Engine eigentlich tut

Ein großes Sprachmodell (Large Language Model, LLM) ist im Kern eine sehr lange Kette von Rechenvorschriften, die aus einem Stück Text eine Wahrscheinlichkeitsverteilung über das nächste Textstück berechnet. Diese Textstücke heißen Token. Das Modell sagt immer nur ein einziges Token voraus; um eine ganze Antwort zu schreiben, hängt die Software das gewählte Token an den Text an und fragt das Modell erneut – Token für Token, bis ein Ende-Zeichen kommt. Diese Arbeitsweise heißt autoregressiv.

FACHBEGRIFF: Token

Die Einheit, in der ein Sprachmodell Text sieht. Ein *Tokenizer* zerlegt Text in häufige Zeichenfolgen: kurze Wörter sind ein Token, lange Wörter werden in mehrere Stücke zerlegt („Inferenz“ etwa in „Inf“, „eren“, „z“). Jedes Token hat eine Nummer, die *Token-ID*. Als Faustregel entspricht ein Token etwa drei Vierteln eines englischen Wortes. Geschwindigkeiten gibt man in Token pro Sekunde (Tok/s) an.

Die Rechenvorschriften selbst sind überwiegend Matrixmultiplikationen. Die Zahlen in diesen Matrizen heißen Gewichte oder Parameter; sie wurden beim Training gelernt und ändern sich bei der Benutzung nicht mehr. Ein Modell mit 125 Milliarden Parametern besteht also aus 125 Milliarden solcher Zahlen. Die Software, die ein fertig trainiertes Modell ausführt, nennt man Inferenz-Engine. Sie lädt die Gewichte, verwaltet den Speicher, ruft für jede Rechenvorschrift den passenden Rechenkern (Kernel) auf der Grafikkarte oder dem Prozessor auf und bietet meist eine Netzwerkschnittstelle an, über die Chat-Programme oder Programmier-Agenten das Modell ansprechen. Fast alle Engines sprechen dafür das Protokoll der OpenAI-API (/v1/chat/completions), so auch Strata, das zusätzlich die Messages-API von Anthropic versteht.

FACHBEGRIFF: Schicht (Layer)

Ein Sprachmodell ist ein Stapel gleich aufgebauter Bausteine. Jede Schicht nimmt für jedes Token einen Zahlenvektor (den *Hidden State*, hier einige tausend Zahlen) entgegen, verrechnet ihn mit den Vektoren der vorherigen Token und mit ihren eigenen Gewichten und reicht das Ergebnis an die nächste Schicht weiter. Das hier besprochene Modell hat 48 Schichten. Am Ende rechnet der *Ausgabekopf* (Output Head) den Vektor in eine Wahrscheinlichkeit für jedes Token des Vokabulars um.

Zwei Phasen: Prompt lesen und Antwort schreiben

Jede Anfrage durchläuft zwei Phasen mit völlig unterschiedlichem Charakter. In der ersten, dem *Prefill*, liest das Modell den gesamten Eingabetext – die Systemanweisung, den bisherigen Gesprächsverlauf, bei Programmier-Agenten oft zehntausende Zeilen Quelltext. Weil alle diese Token schon vorliegen, kann die Engine sie in großen Blöcken parallel durch die Schichten schicken. Jedes Gewicht, das sie aus dem Speicher holt, wird dann für tausende Token gleichzeitig verwendet. Diese Phase ist durch die Rechenleistung begrenzt, und Grafikkarten haben davon reichlich.

In der zweiten Phase, dem *Decode*, entsteht die Antwort Token für Token. Für jedes einzelne neue Token muss die Engine alle benötigten Gewichte erneut aus dem Speicher lesen, rechnet damit aber nur eine winzige Matrix-Vektor-Multiplikation. Die Recheneinheiten warten fast die ganze Zeit auf Daten. Die Geschwindigkeit der Antwort hängt deshalb nahezu ausschließlich davon ab, wie viele Bytes pro Token gelesen werden müssen und wie schnell der Speicher sie liefert.

FACHBEGRIFF: Speicherbandbreite

Die Datenmenge, die ein Speicher pro Sekunde liefern kann. Typische Werte: Grafikspeicher (VRAM) einer RTX 3090 rund 936 GByte/s, einer RTX 5070 rund 670 GByte/s; DDR5-Arbeitsspeicher eines Desktop-PCs mit zwei Kanälen 40 bis 60 GByte/s; der PCIe-4.0-Bus zwischen Hauptplatine und Grafikkarte etwa 26 GByte/s nutzbar; eine NVMe-SSD einige GByte/s. Beim Decode gilt grob: Token pro Sekunde $\approx$ Bandbreite geteilt durch Bytes pro Token.

Eine einfache Rechnung zeigt das Problem. Läge ein Modell mit 66 GByte komplett im Arbeitsspeicher eines PCs, der 50 GByte/s schafft, und müsste für jedes Token vollständig gelesen werden, käme man auf weniger als ein Token pro Sekunde. Dass Strata ein Vielfaches davon erreicht, liegt an zwei Dingen: an einem Modell, das pro Token nur einen kleinen Bruchteil seiner Gewichte braucht, und an einer Engine, die genau weiß, welche Bytes sie wann wo vorhalten muss.

Was sich das Modell merkt: der KV-Cache

Damit ein Token „weiß“, was vorher im Text stand, verrechnen klassische Sprachmodelle es in jeder Schicht mit allen vorherigen Token. Dieser Mechanismus heißt Attention. Damit die Engine die Vorgänger nicht bei jedem neuen Token komplett neu berechnen muss, speichert sie pro Token und Schicht zwei Vektoren, Key und Value. Dieser Zwischenspeicher wächst mit jedem Token und ist bei langen Texten ein ernsthafter Kostenfaktor.

FACHBEGRIFF: KV-Cache

Der Speicher für die Key- und Value-Vektoren aller bisher gelesenen Token. Er macht das Weiterschreiben billig, wächst aber linear mit der Textlänge (dem Kontext). Bei Kontexten von 100 000 Token und mehr belegt er bei klassischen Modellen schnell zweistellige Gigabyte-Beträge – Platz, der dann für Gewichte fehlt. Engines speichern ihn deshalb oft mit reduzierter Genauigkeit, etwa als 8-Bit-Ganzzahlen ($-\text{kv_int8}$).

Der Kontext ist die Länge des Textes, den das Modell auf einmal „im Blick“ hat. Das hier betrachtete Modell beherrscht 262 144 Token, genug für ein mittelgroßes Softwareprojekt samt Gesprächsverlauf. Genau diese Länge braucht ein Programmier-Agent wie Claude Code, OpenCode oder Codex, wenn er selbstständig Dateien liest, Befehle ausführt und Code ändert: Jede Runde schickt er den gesamten bisherigen Verlauf erneut an die Engine.

Gewichte schrumpfen: Quantisierung

Trainiert werden Modelle mit 16-Bit-Gleitkommazahlen (BF16), zwei Byte pro Parameter. 125 Milliarden Parameter wären so 250 GByte. Für die Inferenz komprimiert man die Gewichte deshalb auf zwei bis acht Bit pro Wert – ein Verfahren, das Quantisierung heißt. Je weniger Bit, desto kleiner und schneller, aber desto ungenauer wird das Modell.

FACHBEGRIFF: Quantisierung und GGUF

Statt jedes Gewicht als 16-Bit-Zahl zu speichern, fasst man z. B. 32 Gewichte zu einem *Block* zusammen, speichert einen gemeinsamen Skalierungsfaktor und für jedes Gewicht nur eine kleine Ganzzahl. $Q8_0$ nutzt 8 Bit pro Wert, $Q4_K$ etwa 4,5 Bit, $Q2_0$ 2,25 Bit. Die *K-Quants* ($Q4_K$, $Q5_K$, $Q6_K$) verschachteln Blöcke mit eigenen Unterskalen; die *i-Quants* ($IQ2_XS$, $IQ3_XS$, $IQ3_S$) kodieren Achtergruppen von Gewichten als Index in ein Codebuch von Gitterpunkten und kommen so mit sehr wenigen Bits aus. All diese Formate stammen aus dem Projekt llama.cpp; ihr Containerformat heißt *GGUF*. Eine GGUF-Datei enthält Gewichte, Tokenizer und Metadaten; große Modelle werden auf mehrere *Shards* verteilt.

Wichtig für das Verständnis dieses Artikels: Ein und dasselbe Modell existiert in vielen Quantisierungen verschiedener Anbieter, die sich in Größe, Qualität und den verwendeten Formaten unterscheiden. Eine Engine muss für jedes Format, das in einer Datei vorkommt, passende Rechenkerne mitbringen – auf der GPU und auf der CPU. Fehlt einer, kann sie die Datei nicht oder nur langsam ausführen. Genau an dieser Stelle hat der Fork einen seiner ersten Beiträge geleistet.

Das Modell: 125 Milliarden Parameter, 6 Milliarden pro Token

Strata ist keine Engine für beliebige Modelle, sondern für genau eine Modellfamilie: Qwen3.8-Flash-Next des Qwen-Teams und deren Ableger, etwa das Feintuning Swift 1.5 von UkisAI oder die auf Programmierung beschnittene Coder-Variante von ISTA-DASLab. Diese Spezialisierung ist kein Mangel, sondern das Konstruktionsprinzip – denn der Aufbau dieses Modells ist ungewöhnlich, und Strata nutzt jede seiner Eigenheiten aus. Die Entwickler des Modells haben nach eigener Aussage jede Architekturentscheidung danach bewertet, wie viele Bytes pro Token bewegt werden müssen. Das merkt man an vier Stellen.

Mixture of Experts: ein Team aus 24 576 Spezialisten

Der größte Teil eines klassischen Sprachmodells steckt in den sogenannten Feed-Forward-Netzen, die in jeder Schicht nach der Attention folgen. Qwen3.8-Flash-Next ersetzt dieses eine große Netz pro Schicht durch 512 kleinere Netze, die Experten. Ein winziges Zusatznetz, der Router, entscheidet für jedes Token, welche zehn dieser 512 Experten es bearbeiten. Hinzu kommt ein „geteilter“ Experte, den jedes Token durchläuft. Bei 48 Schichten ergibt das $48 \times 512 = 24\,576$ Experten, von denen pro Token nur 480 arbeiten – rund zwei Prozent.

FACHBEGRIFF: Mixture of Experts (MoE)

Ein Modellaufbau, bei dem jede Schicht viele parallele Teilnetze (Experten) besitzt, von denen ein Router pro Token nur wenige auswählt. Das Modell hat dadurch sehr viele Parameter (Wissen), braucht pro Token aber nur einen kleinen Teil davon (Rechenaufwand). Qwen3.8-Flash-Next hat 125 Milliarden Parameter, aktiviert aber nur etwa 6 Milliarden pro Token. Jeder Experte besteht aus drei Matrizen: *gate* und *up* (zusammen „gate/up“) sowie *down*. Welche Experten ein Token braucht, steht erst fest, wenn der Router der jeweiligen Schicht gerechnet hat.

Für eine Inferenz-Engine ist das zugleich Segen und Fluch. Segen, weil pro Token nur ein Bruchteil der Gewichte gelesen werden muss: In der 2-Bit-Fassung sind das 0,66 GByte Expertengewichte von insgesamt 34 GByte. Fluch, weil man vorher nicht weiß, welcher Bruchteil es sein wird. Alle Experten

müssen griffbereit liegen, obwohl fast alle in jedem Schritt ungenutzt bleiben. Hinzu kommt: Die Wahl ist ungleich verteilt. Manche Experten werden weit häufiger gewählt als andere, und welche das sind, hängt auch vom Thema ab – ein Gespräch über C++ ruft andere Spezialisten auf als eines über Lyrik. Strata baut darauf einen adaptiven Cache, der weiter unten beschrieben ist.

Gated DeltaNet: ein Gedächtnis fester Größe

Von den 48 Schichten benutzen nur 12 eine Form von Attention. Die übrigen 36 sind sogenannte Gated-DeltaNet-Schichten, eine moderne Form rekurrenter Netze. Statt sich jedes vergangene Token einzeln zu merken, pflegen sie einen Zustand fester Größe, den sie mit jedem Token fortschreiben – vergleichbar mit einem Notizblock, auf dem laufend radiert und neu geschrieben wird, statt eines Archivs, das endlos wächst.

FACHBEGRIFF: Rekurrenter Zustand

Eine rekurrente Schicht fasst die gesamte Vorgeschichte in einem Zustand fester Größe zusammen, der sich mit jedem Token ändert. Das spart enorm Speicher bei langen Texten. Der Preis: Man kann nicht beliebig „zurückspulen“. Um den Zustand nach Token 5000 zu erhalten, muss man ihn entweder an dieser Stelle gespeichert haben (Checkpoint) oder alle 5000 Token erneut verarbeiten. Diese Eigenschaft wird beim Wiederaufnehmen von Gesprächen eine zentrale Rolle spielen.

Die zwölf übrigen Schichten nutzen *Qwen Sparse Attention* (QSA). Sie hat nur zwei Key-Value-Köpfe, was den KV-Cache klein hält, und betrachtet pro Token höchstens rund 2048 ausgewählte frühere Positionen, ganz gleich, wie lang der Text ist. Ein kleiner Indexer wählt diese Positionen aus. Für die Engine heißt das: Der Aufwand pro Token wächst mit der Kontextlänge nur sehr langsam, und der KV-Cache für 262 000 Token bleibt im einstelligen Gigabyte-Bereich.

Eine riesige Nachschlagetabelle

Die dritte Eigenheit ist eine 51 Milliarden Parameter große Tabelle von N-Gramm-Embeddings (im Code PLE für Per-Layer Embeddings). Die drei zuletzt gelesenen Token-IDs bilden einen Schlüssel, unter dem jede Schicht eine zusätzliche Zeile Information nachschlägt. In der gebräuchlichsten Quantisierung belegt die Tabelle 28,8 GByte, pro Token werden aber nur 16 Zeilen von zusammen höchstens 23 KByte gebraucht. Und weil der Schlüssel aus bereits bekannten Token besteht, weiß die Engine schon vor der Berechnung, welche Zeilen sie benötigt. Eine solche Tabelle kann daher auf der SSD liegen: Die Engine fordert die Zeilen einen Schritt im Voraus an, das Betriebssystem hält häufige Zeilen in seinem Dateicache.

Ein eingebauter Vorhersager

Die vierte Eigenheit ist eine zusätzliche Schicht für Multi-Token-Prediction (MTP). Sie ist darauf trainiert, aus dem Zustand des Hauptmodells die nächsten Token billig zu erraten. Strata nutzt sie für spekulatives Dekodieren, das weiter unten erklärt wird. Die gängigen GGUF-Dateien enthalten diese Schicht nicht; Strata holt ihre Tensoren deshalb gezielt aus dem originalen BF16-Checkpoint auf Hugging Face (rund 5 GByte statt 250) und komprimiert sie selbst.

Ergänzt wird der Aufbau durch *Hyper-Connections*, eine

Tabelle 1. Wo die Bytes stecken (2-Bit-Fassung Q2_0, nach dem Strata-Paper)

Bestandteil	gespeichert	pro Token gelesen
Geroutete Experten (48 × 512)	34,0 GB	0,66 GB (2 %)
Mixer, Attention, geteilte Experten, Router	1,8 GB	1,8 GB
Hyper-Connections	1,3 GB	1,3 GB
Ausgabekopf	0,44 GB	0,44 GB
N-Gramm-Tabelle (PLE)	28,8 GB	≤ 23 KB
MTP-Schicht	0,8 GB	je Entwurf

Variante der Restverbindungen zwischen den Schichten mit eigenen, kleinen Gewichtsmatrizen. Tabelle 1 zeigt, wo die Bytes der 2-Bit-Fassung stecken – und was davon pro Token tatsächlich gelesen wird.

Die Tabelle enthält die zentrale Einsicht, auf der Strata aufbaut: Es gibt einen kleinen, immer benötigten Teil von rund 3,5 GByte (Mixer, Router, Hyper-Connections, Ausgabekopf) und einen riesigen, selten und unvorhersehbar benötigten Teil (die Experten und die Tabelle). Der kleine Teil gehört auf die Grafikkarte, der große muss irgendwo anders liegen – und dort möglichst trotzdem schnell erreichbar sein.

Die Quantisierungen im Überblick

Für Qwen3.8-Flash-Next existieren mehrere Familien von GGUF-Dateien, die Strata ausführen kann:

GSQ-RCO von ISTA-DASLab: speziell optimierte Quantisierungen in Q2_0 (37,6 GB), IQ2_XS (39,2 GB), IQ3_XXS (47 GB) und IQ3_S (54,8 GB). Auf sie war Strata ursprünglich ausgelegt; sie passen in 64 GB Arbeitsspeicher.

Unsloth UD-Q4_K_XL: eine „dynamische“ 4-Bit-Quantisierung mit gemischten Formaten – Q4_K und Q5_K für gate/up, Q5_1 und Q8_0 für down, Q8_0 für Embedding und Ausgabekopf, IQ4_NL für die N-Gramm-Tabelle. 111 GB auf vier Shards, die Experten allein 71,7 GiB. Dem Original am nächsten, aber für 64-GB-PCs zu groß.

Feintunings: Swift 1.5 Q4_K_L (mit Q5_0-Experten), OrcaRouter Q4_K_S, Uncensored Q5_K_M (mit einer Q5_1-Tabelle), Unsloth UD-Q6_K_XL (Q6_K-Experten). Jede bringt Formatkombinationen mit, für die eine Engine eigene Rechenkerne braucht.

Der Fork arcfork-strata konzentriert sich auf die zweite und dritte Gruppe: die „gewöhnlichen“ GGUF-Dateien, wie sie auf Hugging Face für jedes Modell erscheinen.

Die Platzhirsche: llama.cpp und vLLM

Wer heute ein offenes Sprachmodell selbst betreibt, landet fast zwangsläufig bei einem von zwei Projekten. Beide sind hervorragend – aber für andere Situationen gebaut als die, in der Strata glänzt.

llama.cpp: der Generalist für jede Hardware

llama.cpp entstand 2023 als Projekt, Metas LLaMA-Modelle auf einem MacBook laufen zu lassen, und ist heute die Referenz für lokale Inferenz. Es ist in C und C++ geschrieben, baut auf der hauseigenen Tensorbibliothek ggml auf und läuft auf nahezu allem: x86- und ARM-Prozessoren, NVIDIA-Karten über CUDA, AMD über HIP, Apple über Metal, dazu Vulkan und weitere Backends. Es unterstützt hunderte Modellarchitekturen. Das GGUF-Format und fast alle verbreiteten Quantisierungsformate stammen aus diesem Projekt; Werkzeuge wie Ollama oder LM Studio bauen darauf auf.

Passt ein Modell nicht in den Videospeicher, verteilt llama.cpp es: Ein Teil der Schichten liegt auf der GPU, der Rest im Arbeitsspeicher und wird dort vom Prozessor gerechnet. Für MoE-Modelle hat sich eingebürgert, per Kommandozeile (`--override-tensor` oder `--n-cpu-moe`) gezielt die großen Experten-Tensoren in den Arbeitsspeicher zu legen und alles andere auf die GPU. Das funktioniert gut und ist ein großer Fortschritt gegenüber reiner CPU-Inferenz. Die Verteilung ist aber statisch und grob: Sie wird beim Laden festgelegt, und zwar pro Tensor. In GGUF-Dateien liegen alle 512 Experten einer Schicht in einem gemeinsamen Tensor. Entweder liegen also alle Experten einer Schicht auf der GPU oder alle im RAM; dass ein Gespräch über Rust gerade andere Experten braucht als eines über SQL, spielt keine Rolle. Und während der Prozessor seine Experten rechnet, wartet die Grafikkarte – und umgekehrt.

vLLM: der Durchsatz-Spezialist fürs Rechenzentrum

vLLM kommt aus der Forschung an der UC Berkeley und ist der De-facto-Standard, wenn ein Modell viele Nutzer gleichzeitig bedienen soll. Seine Kernidee, *PagedAttention*, verwaltet den KV-Cache wie ein Betriebssystem seinen Arbeitsspeicher: in kleinen Seiten, die sich flexibel verschiedenen Anfragen zuteilen lassen. Zusammen mit *Continuous Batching* kann vLLM so dutzende oder hunderte Anfragen gleichzeitig verarbeiten und die Gewichte, die ohnehin für jedes Token gelesen werden müssen, für alle gleichzeitig nutzen.

FACHBEGRIFF: Batching

Werden mehrere Anfragen gleichzeitig durch das Modell geschickt, liest die Engine jedes Gewicht nur einmal und rechnet es mit den Vektoren aller Anfragen. Da beim Decode das Lesen der Gewichte die Zeit bestimmt, steigt der Gesamtdurchsatz fast linear mit der Zahl der Anfragen. *Continuous Batching* nimmt neue Anfragen auf, sobald alte fertig sind, statt auf das Ende eines ganzen Stapels zu warten. Für einen einzelnen Nutzer bringt Batching dagegen nichts.

vLLM ist in Python auf PyTorch-Basis geschrieben, mit hochoptimierten CUDA-Kernen für die heißen Pfade. Es lädt Modelle typischerweise im Safetensors-Format von Hugging Face und unterstützt Quantisierungen wie AWQ, GPTQ, FP8 oder NVFP4. Mehrere Grafikkarten bindet es per *Tensor-Parallelität* ein. Die Grundannahme lautet dabei: Die Gewichte liegen im Videospeicher. Zwar gibt es eine Option, einen Teil in den Arbeitsspeicher auszulagern, doch dann werden diese Gewichte bei jedem Durchlauf über den PCIe-Bus kopiert – für ein Modell, das fünfmal größer ist als der Videospeicher, ein Ausschlusskriterium. Ein 125-Milliarden-Modell in vier Bit braucht unter vLLM mehrere Rechenzentrums-GPUs.

FACHBEGRIFF: Tensor- vs. Pipeline-Parallelität

Zwei Arten, ein Modell auf mehrere GPUs zu verteilen. Bei der *Tensor-Parallelität* wird jede Gewichtsmatrix zerschnitten; alle GPUs rechnen an jeder Schicht mit und tauschen pro Schicht ihre Teilergebnisse aus – schnell mit NVLink, teuer über PCIe. Bei der *Pipeline-* oder *Schichtparallelität* (Layer Split) bekommt jede GPU einen zusammenhängenden Block von Schichten; die Daten wandern nur an den Blockgrenzen von Karte zu Karte. Strata nutzt die zweite Variante: ein paar hundert Kilobyte pro Durchlauf, ohne NVLink.

Die Lücke, in die Strata stößt

Damit ist das Feld abgesteckt. llama.cpp kann riesige MoE-Modelle auf kleiner Hardware ausführen, lässt aber einen Großteil der Maschine die meiste Zeit untätig. vLLM nutzt Grafikkarten exzellent aus, wenn das Modell hineinpasst und viele Nutzer warten. Für das Szenario „ein riesiges MoE-Modell, ein oder wenige Nutzer, Consumer-Hardware“ ist keines der beiden gebaut. Genau hier setzt Strata an. Der Ansatz ist nicht völlig neu – Forschungsprojekte wie Fiddler, KTransformers oder MoE-Infinity haben die Rechenteilung zwischen CPU und GPU sowie Experten-Caches untersucht, und die Idee „eine Engine für ein Modell“ stammt von Projekten wie Splash, ninfer und HyperQwen. Strata führt diese Ideen für ein konkretes Modell konsequent zusammen und ergänzt sie um eigene.

Bemerkenswert ist, wie viel Strata dabei von llama.cpp übernimmt: die Quantisierungsformate, die GPU-Rechenkerne für die i-Quants (aus llama.cpps CUDA-Code übertragen), die MMQ-Kernel für quantisierte Matrixmultiplikationen, das CPU-Backend von ggml für die Expertenberechnung auf dem Prozessor und die Python-Bibliothek gguf-py zum Lesen der Dateien. Strata ist also weniger Konkurrent als Spezialaufbau auf dem Fundament von llama.cpp – eine Art Rennwagen, der Motor und Getriebe aus der Serie übernimmt, aber Fahrwerk und Aerodynamik für genau eine Strecke neu konstruiert.

Strata: eine Engine, maßgeschneidert für ein Modell

Strata ist ein Open-Source-Projekt auf GitHub (Niko1221/Strata, MIT-Lizenz) und noch keine zwei Wochen alt – die erste Version erschien am 24. September 2026, Anfang Oktober ist bereits Version 0.1.37 aktuell. In dieser kurzen Zeit hat das Projekt eine erstaunliche Reife erreicht: Ein-Klick-Installer für Windows und Linux, eine Web-Oberfläche mit Chat und Live-Monitor, Bildverarbeitung, Werkzeugaufrufe, Unterstützung für AMD-Karten und für mehrere GPUs. Der Kern ist ein rund 87 000 Zeilen umfassendes Geflecht aus C++, CUDA und Python. Sein Grundgedanke lässt sich in einem Satz fassen: Jedes Bauteil des PCs bekommt die Arbeit, die es am besten kann – und alle arbeiten gleichzeitig.

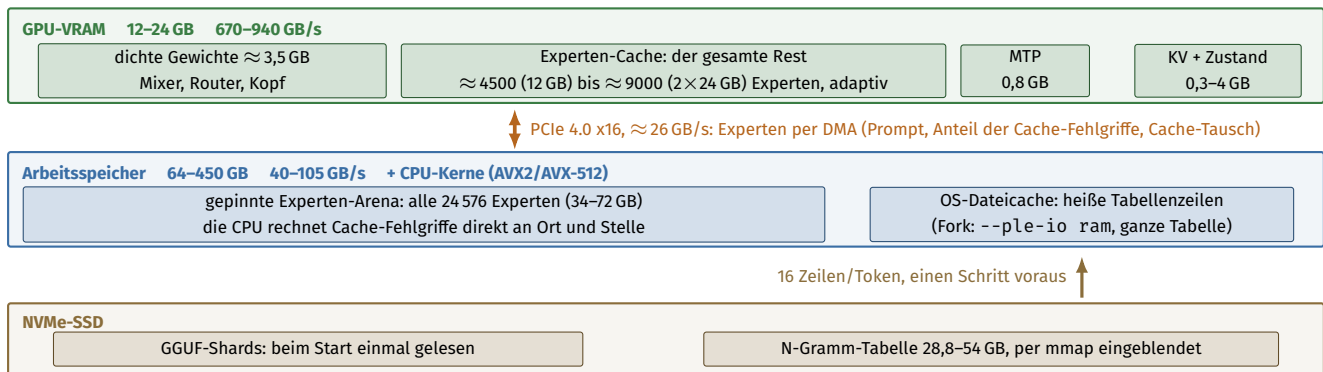
Drei Speicherebenen

Abbildung 1 zeigt, wie Strata das Modell verteilt. Auf der Grafikkarte liegt alles, was jedes Token braucht: die DeltaNet- und Attention-Schichten, die Router, die geteilten Experten, die Hyper-Connections, der Ausgabekopf, die MTP-Schicht sowie KV-Cache und rekurrente Zustände. Das sind je nach Kontextlänge 4 bis 7 GByte. Den gesamten restlichen Videospeicher füllt ein *Experten-Cache* mit den meistgenutzten Experten – auf einer 12-GByte-Karte rund 4500 der 24 576.

Im Arbeitsspeicher liegen *alle* Experten, in einem einzigen großen, „gepinnten“ Speicherbereich, der Arena. Gepinnt heißt: Das Betriebssystem darf diese Seiten nicht auslagern oder verschieben, sodass die Grafikkarte direkt per DMA darauf zugreifen kann, ohne dass der Prozessor Daten umkopieren muss.

Tabelle 2. Drei Engines, drei Philosophien

	llama.cpp	vLLM	Strata / arcfork-strata
Ziel	jedes Modell auf jeder Hardware	maximaler Durchsatz für viele Nutzer	ein Modell, maximale Geschwindigkeit für einen bis wenige Nutzer auf Consumer-Hardware
Modelle	hunderte Architekturen	sehr viele, v. a. Safetensors	nur Qwen3.8-Flash-Next und Ableger
Hardware	CPU, CUDA, HIP, Metal, Vulkan u. a.	primär Rechenzentrums-GPUs (CUDA, ROCm)	NVIDIA ab RTX 20 (CUDA); Upstream zusätzlich AMD (HIP); Fork: Linux, NVIDIA
Modell größer als VRAM	statische Aufteilung pro Tensor/Schicht; CPU und GPU wechseln sich ab	praktisch nicht vorgesehen	alle Experten im RAM, häufige zusätzlich in VRAM; CPU, GPU und PCIe arbeiten gleichzeitig
MoE-Experten	ganze Experten-Tensoren je Schicht fest auf GPU oder CPU	alle in VRAM	einzelne Experten dynamisch im VRAM-Cache, nachgeführt nach Gesprächsthema
Mehrere GPUs	Schicht- oder Zeilenaufteilung	Tensor- und Pipeline-Parallelität	Schichtaufteilung, eine Übergabe pro Durchlauf
Parallele Anfragen	mehrere Slots, Batching	Continuous Batching, PagedAttention	Upstream: eine zur Zeit; Fork: zwei im gemeinsamen Verify-Fenster (Batch-2)
Spekulatives Dekodieren	mit separatem Entwurfsmodell	u. a. mit Entwurfsmodellen und MTP	mit der eingebauten MTP-Schicht, exakt
Gespräche wiederaufnehmen	Prompt-Cache pro Slot	Prefix-Caching des KV-Cache	Checkpoints des rekurrenten Zustands; Fork: Parken mehrerer Gespräche auch bei Schichtaufteilung

**Abbildung 1.** Die drei Speicherebenen von Strata. Was jedes Token braucht, liegt auf der GPU; alle Experten liegen im Arbeitsspeicher, die meistgenutzten zusätzlich im VRAM; die große Nachschlagetabelle bleibt auf der SSD oder – im Fork – komplett im RAM.**FACHBEGRIFF: Pinned Memory und DMA**

Normaler Arbeitsspeicher kann vom Betriebssystem jederzeit verschoben oder auf die Platte ausgelagert werden. Eine Grafikkarte kann ihn deshalb nicht direkt lesen; der Treiber kopiert Daten zuerst in einen festgesetzten (gepinnten, page-locked) Zwischenpuffer. Pinnt man den Speicher von vornherein, kann die *Copy Engine* der GPU ihn per *Direct Memory Access* (DMA) selbstständig über PCIe lesen, während Prozessor und Rechenkerne der GPU anderes tun. Unter Windows (Treibermodell WDDM) lässt sich nur begrenzt Speicher pinnen, unter Linux praktisch beliebig viel.

Auf der SSD schließlich bleibt die N-Gramm-Tabelle. Strata blendet sie per mmap in den Adressraum ein und liest die 16 benötigten Zeilen pro Token einen Schritt im Voraus, sodass sie bereitliegen, wenn die Schicht sie braucht. Den Rest erledigt der Dateicache des Betriebssystems.

Eine Schicht, drei Arbeiter gleichzeitig

Das Herzstück ist der Ablauf innerhalb einer einzelnen Schicht während des Decodes (Abbildung 2). Die GPU rechnet zuerst den Mixer (DeltaNet oder Attention) und den Router. Die zehn gewählten Experten-IDs pro Token schreibt sie in einen kleinen Bereich gepinnten Speichers, den das Projekt „Türklingel“ (Doorbell) nennt. Ein CPU-Thread fragt diese Stelle in einer engen Schleife ab, statt auf eine Benachrichti-

gung des Treibers zu warten – das spart wertvolle Mikrosekunden.

Sobald die IDs da sind, sortiert die CPU die Experten in drei Gruppen:

- Experten, die bereits im VRAM-Cache liegen, rechnet die GPU.
- Ein einstellbarer Anteil der übrigen (`--pcie-frac`) wird per DMA über PCIe auf die GPU kopiert und dort gerechnet, sobald er angekommen ist.
- Den Rest rechnen die CPU-Kerne direkt im Arbeitsspeicher, mit handgeschriebenen AVX-512- und AVX2-Kernen oder den Vektorroutinen von `ggml`.

Die CPU schreibt ihre Ergebnisse in Speicher, den die GPU direkt sehen kann; die GPU addiert alle Beiträge und beginnt die nächste Schicht. Damit arbeiten GPU, CPU und PCIe-Bus in jeder Schicht gleichzeitig. Die nächste Schicht wartet lediglich auf den langsamsten der drei.

Damit der Prozessor die Grafikkarte nicht mit tausenden einzelnen Befehlen pro Token ausbremst, ist der komplette Durchlauf durch alle 48 Schichten als *CUDA Graph* aufgezeichnet – ein Graph pro Fenstergröße. Der Prozessor stößt

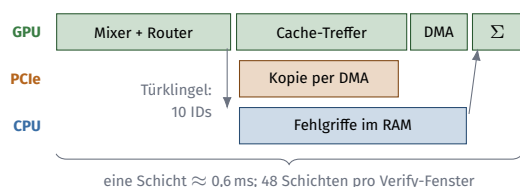


Abbildung 2. Ablauf einer Schicht: Die GPU rechnet ihre Cache-Treffer, während die CPU die fehlenden Experten im RAM rechnet und die Copy Engine einen Teil über PCIe holt.

ihn mit einem einzigen Aufruf an.

FACHBEGRIFF: CUDA Graph

Normalerweise schickt die CPU jeden Rechenkern einzeln an die GPU, was pro Aufruf einige Mikrosekunden kostet. Bei hunderten Kernen pro Token summiert sich das. Ein CUDA Graph zeichnet eine ganze Abfolge von Kernen, Kopien und Abhängigkeiten einmal auf; danach startet ein einziger Aufruf die ganze Folge. Nachteil: Die Form der Daten (etwa die Zahl der Token im Fenster) ist im Graphen fest, für jede Form braucht es einen eigenen.

Wie viel welcher Arbeiter beiträgt, ist im technischen Paper des Projekts genau vermessen. Auf seinem Testrechner (RTX 5070, Ryzen 5 7600, 64 GByte DDR5) dauert ein Durchlauf mit der 2-Bit-Fassung bei 4000 Token Kontext rund 34 Millisekunden, davon wartet die CPU etwa 15 ms auf die GPU und die GPU etwa 13 ms auf die CPU. Die Engine ist also ausbalanciert – was bedeutet, dass keine einzelne Aufrüstung die Geschwindigkeit verdoppeln kann. Bei der 2-Bit-Fassung begrenzt die Speicherbandbreite des RAM (die CPU streamt Gewichte mit rund 42 GByte/s, nahe am Maximum), bei den i-Quants die Rechenleistung der CPU, weil das Entschlüsseln der Codebuch-Gewichte mehrere Befehle pro Wert kostet.

Lehrreich sind auch die Versuche, die nicht funktioniert haben. Ein Fenster in zwei Token-Gruppen zu teilen, damit die CPU-Experten der einen Gruppe parallel zur GPU-Arbeit der anderen laufen, war zwar exakt, aber 7 Prozent *langsamer*: Die dichten Gewichte mussten zweimal gelesen werden. Was dagegen half, war die Copy Engine. Ursprünglich holte ein Rechenkern der GPU die fehlenden Experten selbst über PCIe – das lag auf dem kritischen Pfad. Seit die CPU die Kopien als DMA-Aufträge erteilt, sobald sie die Schicht plant, laufen sie neben allem anderen. Bei den i-Quant-Dateien gehen so 55 Prozent der Fehlgriffe über PCIe, was 10 bis 16 Prozent Geschwindigkeit brachte.

Ein Cache, der dem Gespräch folgt

Beim Start füllt Strata den VRAM-Cache anhand eines Profils, das auf anderen Texten aufgezeichnet wurde: Die meistgewählten Experten kommen zuerst. Ein solches statisches Profil trifft bei 4500 Plätzen etwa die Hälfte aller Expertenaufrufe. Während des Gesprächs zählt die Engine dann mit, welche Experten tatsächlich gerufen werden, und tauscht alle vier Durchläufe bis zu 96 selten genutzte gegen häufig verfehlte aus – während die GPU gerade mit dem Entwerfen beschäftigt ist. Damit steigt die Trefferquote auf rund 72 Prozent. Zwei Details zeigen, wie genau hier gearbeitet wird: Der alte Experte wird sofort verdrängt, der neue aber erst freigeschaltet, wenn seine Kopie angekommen ist, sodass niemand wartet (plus 3 Prozent). Und die Plätze im Cache sind

1. MTP-Schicht rät drei Token



2. Hauptmodell prüft alle in einem Durchlauf



Ergebnis: drei Token aus einem Durchlauf – zwei bestätigte Entwürfe plus das eigene Token des Hauptmodells.

Abbildung 3. Spekulatives Dekodieren: Die eingebaute MTP-Schicht entwirft, das volle Modell prüft. Der Text ist identisch mit dem, den das Modell allein geschrieben hätte.

pro Schicht genau so groß wie deren Experten, statt alle auf den größten auszulegen – bei IQ3_XXS schwanken Experten zwischen 1,3 und 2,3 MByte, was 33 Prozent mehr Plätze und 13 Prozent mehr Tempo brachte.

FACHBEGRIFF: Cache-Trefferquote

Der Anteil der Expertenaufrufe, die aus dem VRAM bedient werden. Jeder Treffer ist ein Experte, den die langsame CPU nicht rechnen muss. Faustregel aus Stratas Messungen: Jedes zusätzliche Gigabyte VRAM fasst etwa 700 weitere Experten der 2-Bit-Fassung. Deshalb ist bei Strata die *Größe* des Videospeichers wichtiger als die Rechenleistung der Karte.

Raten und prüfen: spekulatives Dekodieren

Selbst wenn alle Experten im Cache lägen, bliebe ein fester Preis pro Durchlauf: Die 3,5 GByte dichter Gewichte müssen gelesen werden. Dieser Preis steigt kaum, wenn statt einem Token vier gleichzeitig durch das Modell laufen. Hier setzt spekulatives Dekodieren an (Abbildung 3).

Die MTP-Schicht entwirft bis zu drei Folgetoken; zusammen mit dem letzten bestätigten Token laufen sie als *Verify-Fenster* gemeinsam durch alle 48 Schichten. Das Hauptmodell berechnet dabei für jede Position, welches Token es selbst gewählt hätte. Stimmt der Entwurf, wird er übernommen; beim ersten Widerspruch wird der Rest verworfen und stattdessen das Token des Hauptmodells genommen. So entstehen pro Durchlauf im Mittel 2,4 bis 3,2 Token. Ein Entwurf kommt nur ins Fenster, wenn die MTP-Schicht sich zu mindestens 50 Prozent sicher ist (--spec-min-p 0.5). Wiederholt eine Antwort Text aus dem Kontext – typisch bei Code-Änderungen –, schlägt zusätzlich ein *Prompt Lookup* bis zu fünf Token aus der früheren Fundstelle vor.

FACHBEGRIFF: Spekulatives Dekodieren

Ein billiges Verfahren rät mehrere Folgetoken, das teure Modell prüft alle Vermutungen in einem einzigen Durchlauf. Da das Lesen der Gewichte die Zeit bestimmt und nicht das Rechnen, kostet das Prüfen von vier Token kaum mehr als das Erzeugen von einem. Richtig umgesetzt ist das Ergebnis *exakt* dasselbe wie ohne Raten: Das große Modell entscheidet jedes Token. Strata testet das mit absichtlich falschen Entwürfen.

Bei Strata bringt das Verfahren den Faktor 1,6 bis 1,8 – weniger als die 2,5 bis 3, die Engines melden, bei denen das ganze Modell im VRAM liegt. Der Grund ist aufschlussreich: Jedes zusätzliche Token im Fenster wählt eigene Experten, die womöglich nicht im Cache liegen und von der CPU gerechnet werden müssen. Die Kosten eines Fensters wachsen also mit seiner Breite, und zwar spürbar.

Prompts lesen: Experten wandern zur GPU

Beim Prefill kehrt sich das Verhältnis um. Hier werden Prompts in Blöcken von 2048 bis 8192 Token gelesen, und bei so vielen Token wird praktisch jeder Experte vielfach gebraucht. Die richtige Stelle zum Rechnen ist jetzt die GPU, auch für Experten, die nicht im Cache liegen. Strata kopiert sie deshalb Schicht für Schicht aus der gepinnten Arena über PCIe, während die GPU bereits die vorherigen rechnet. Die Puffer dafür „leiht“ sich der Prompt-Pfad aus dem Experten-Cache und füllt diesen danach wieder auf.

Gerechnet werden die Experten auf zwei Wegen: Entweder werden sie in 16-Bit-Gleitkommazahlen entpackt und mit der NVIDIA-Bibliothek cuBLAS multipliziert, oder sie laufen quantisiert durch die MMQ-Kernel aus llama.cpp, die direkt auf den komprimierten Daten rechnen. MMQ ist deutlich schneller, braucht aber für jedes Quantisierungsformat eigene Kernel-Varianten. Weil die Sparse Attention nur eine begrenzte Zahl an Positionen betrachtet, bleibt die Prompt-Geschwindigkeit über alle Kontextlängen nahezu konstant.

Lange Kontexte und Gesprächsgedächtnis

Ab 8000 Token speichert Strata den KV-Cache als 8-Bit-Ganzzahlen, ab 64 000 Token hält es nur noch den am häufigsten gelesenen Teil im VRAM und streamt den Rest aus dem RAM. Die MTP-Schicht betrachtet nur die letzten 32 768 Positionen, damit das Raten auch bei 262 000 Token billig bleibt. Der eigentliche Preis langer Kontexte ist Platz: Bei vollem Kontext passen auf eine 12-GB-Byte-Karte statt 4500 nur noch etwa 1600 Experten.

Damit nicht jede Anfrage den kompletten Verlauf neu lesen muss, hält Strata die laufende Sitzung und bis zu sechs *Checkpoints* von je rund 118 MByte im RAM – Momentaufnahmen des rekurrenten Zustands und des KV-Cache, angelegt zu Beginn jeder Antwort und alle 16 000 Prompt-Token. Eine Folgefrage, deren Anfang exakt zu einem Checkpoint passt, liest nur den neuen Teil. Exakt heißt dabei: Token für Token. Wegen der rekurrenten Schichten kann die Engine nicht an eine beliebige Stelle zurückspringen (siehe Kasten „Rekurrenter Zustand“). Seit Ende September kann Upstream zudem mehrere Gespräche im RAM *parken* – allerdings nur auf einer einzelnen GPU.

Mehrere Grafikkarten

Mit zwei oder drei Karten teilt Strata die 48 Schichten in zusammenhängende Blöcke auf. Jede Karte hält eine Kopie der dichten Gewichte, ihren Teil der Zustände und einen Experten-Cache nur für ihre eigenen Schichten; zwei Karten fassen so etwa doppelt so viele Experten. Die letzte Karte rechnet zusätzlich Ausgabekopf und MTP-Schicht. Ein Token wechselt pro Verify-Fenster nur einmal die Karte – ein paar hundert Kilobyte über den gepinnten RAM, ohne NVLink. Die Aufteilung wählt Strata automatisch: Es probiert alle Varianten und nimmt die, bei der die Caches den größten Teil des Expertenprofils fassen.

Genauigkeit als Prinzip

Bemerkenswert an Strata ist der Anspruch an Exaktheit. Jeder aus llama.cpp übernommene oder neu geschriebene Rechenkern wird gegen die Referenzarithmetik von ggml geprüft, auf

echten Schichten der Modelldateien. Das spekulative Dekodieren muss Token für Token denselben Text liefern wie gieriges Einzeldekodieren. Und im Vergleich mit llama.cpp liegt die mittlere KL-Divergenz der Wahrscheinlichkeitsverteilungen bei 0,022 – weniger als die 0,058, um die sich llama.cpps eigene CPU- und GPU-Builds voneinander unterscheiden.

FACHBEGRIFF: KL-Divergenz

Ein Maß dafür, wie stark zwei Wahrscheinlichkeitsverteilungen voneinander abweichen; 0 heißt identisch. Man nutzt es, um zu prüfen, ob eine Engine oder eine Quantisierung dieselben Vorhersagen liefert wie eine Referenz. Unterschiede entstehen schon durch die Reihenfolge, in der Gleitkommazahlen addiert werden.

Auf dem Referenz-PC mit RTX 5070 schreibt die aktuelle Upstream-Version mit der 2-Bit-Fassung 87 bis 93 Token pro Sekunde bei kurzen Kontexten und rund 74 bei 128 000 Token; Prompts liest sie mit gut 2000 Token pro Sekunde. Das ist die starke Basis, auf der der Fork aufsetzt.

Der Fork: arcfork-strata

Upstream-Strata ist auf einen klaren Zielrechner ausgelegt: eine Grafikkarte mit 12 bis 24 GByte, 64 GByte RAM, ein Desktop-Prozessor mit sechs bis sechzehn Kernen, Windows oder Linux, ein Nutzer im Chat. Der Fork arcfork-strata zielt auf eine ganz andere Maschine und eine ganz andere Last. Er läuft auf einem Server mit zwei RTX 3090 (je 24 GByte), einem AMD EPYC 7413 mit 24 Kernen und 450 GByte Arbeitsspeicher, unter Linux mit CUDA 13.3. Und er bedient keine Chat-Nutzer, sondern Programmier-Agenten mit dem vollen Kontext von 262 000 Token – dazu einen Web-Chat und einen Matrix-Bot, die sich die Engine teilen. Eingebunden ist er als GGUF-Backend in eine interne Dienste-Plattform, neben einem unveränderten vLLM für Modelle im Safetensors-Format.

Aus dieser Situation ergeben sich andere Anforderungen. Statt der kleinen GSQ-RCO-Dateien soll die hochwertigere 4-Bit-Fassung UD-Q4_K_XL von Unsloth laufen (103,7 GiB) sowie Feintunings wie Swift-1.5 Q4_K_L, die es nur als gewöhnliche GGUF-Dateien gibt. Prompts sind lang und kommen oft, weil Agenten in jeder Runde ihren ganzen Verlauf schicken. Mehrere Clients wechseln sich ab, sodass die Engine ständig zwischen Gesprächen springt. Und gelegentlich wollen zwei Agenten gleichzeitig arbeiten.

Der Fork begann am 30. September 2026 auf Upstream 0.1.28 und wurde am 1. Oktober auf 0.1.32 nachgezogen. Er umfasst gut 50 eigene Commits; gegenüber Upstream ändert er 38 Dateien mit rund 3600 hinzugefügten Zeilen, zu einem großen Teil in der Steuerlogik der Engine (generate.cpp), im Verify-Fenster, im Prompt-Pfad und in den CUDA-Kernen. Wichtig für die Einordnung: Upstream entwickelt sich parallel rasant weiter. Einiges, was der Fork früh gebaut hat, hat Upstream inzwischen in eigener Form übernommen oder unabhängig entwickelt – etwa die Unterstützung gewöhnlicher GGUF-Experten, die vor allem auf einen anderen Fork (eddoursul/Strata) zurückgeht. Der Fork folgt in solchen Fällen der Upstream-Variante und portiert nur seine eigenen Ergänzungen darauf. Abbildung 4 zeigt das Ergebnis auf der Zielmaschine.

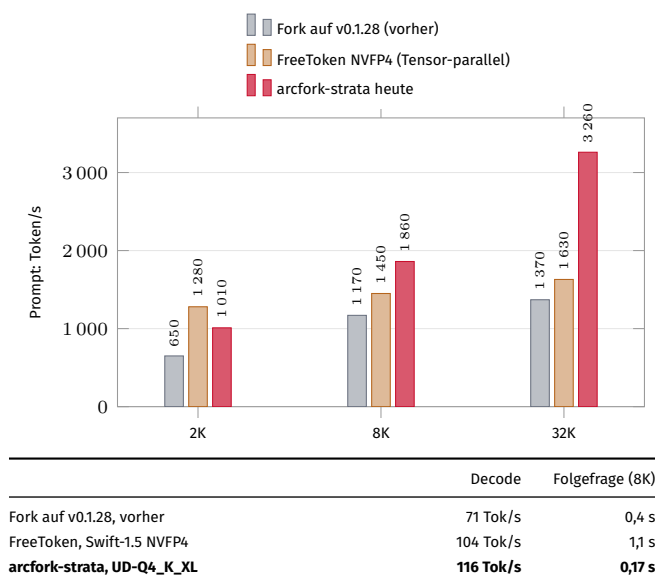


Abbildung 4. Prompt-Geschwindigkeit, Schreibtempo und Antwortzeit auf eine Folgefrage auf 2× RTX 3090, 262K Kontext, Coding-Last. Die Verbesserung gegenüber „vorher“ stammt teils aus Upstream-Versionen 0.1.29–0.1.32, teils aus den Fork-Änderungen.

Die Prompt-Geschwindigkeit bei 32 000 Token hat sich mehr als verdoppelt, das Schreibtempo um gut 60 Prozent erhöht, und eine Folgefrage beginnt mehr als doppelt so schnell. Zum Vergleich steht FreeToken in der Grafik, eine andere auf Consumer-Hardware spezialisierte MoE-Engine, die hier mit einer NVFP4-Fassung per Tensor-Parallelität über beide Karten lief: Bei kurzen Prompts ist sie schneller, ab 8000 Token liegt der Fork vorn, bei 32 000 Token um den Faktor zwei. Wie es dazu kam, zeigen die folgenden Abschnitte.

Gewöhnliche GGUF-Dateien zum Laufen bringen

Die erste Hürde war banal und grundlegend zugleich: Upstream 0.1.28 konnte die Experten von UD-Q4_K_XL schlicht nicht auf der GPU rechnen. Die Engine kannte die i-Quants und Q2_0, aber nicht die K-Quants (Q4_K, Q5_K, Q6_K) für gate/up und nicht Q5_1 und Q8_0 für down. Der Fork ergänzte diese Formate in den gruppierten Decode-Kernen, die mehrere Experten in einem Aufruf rechnen, in der 16-Bit-Entpackung für den Prompt-Pfad (in exakt der Wertereihenfolge, die ggml verwendet) und in den MMQ-Kernen von llama.cpp.

Dazu kamen Eigenheiten der Dateien. Unsloth verteilt die Shards so, dass gate/up und down einer Schicht in verschiedenen Dateien landen können – in UD-Q4_K_XL etwa bei Schicht 11. Das Pack-Werkzeug iq_pack.py verweigerte das, weil die Engine pro Schicht nur eine Datei kannte. Der Fork führte „per-role shards“ ein: pro Schicht drei Dateinamen, sobald sie sich unterscheiden. Das Embedding liegt als Q8_0 vor und ist auf mehrere Shards verteilt; auch dafür brauchte es Anpassungen.

Fork-Beitrag: Ein Fehler im Prüfstand

Die MMQ-Variante für Q5_1 lieferte im Paritätstest nur Nullen, weshalb sie zunächst abgeschaltet blieb – 43 der 48 Schichten von UD-Q4_K_XL liefen dadurch über den langsameren Umweg aus Entpacken und cuBLAS. Die Ursache

lag nicht im Kernel, sondern im Test: Er übergab dem Aktivierungs-Quantisierer Zeiger auf Hauptspeicher statt auf GPU-Speicher und benutzte Puffer, die er bereits freigegeben hatte. Mit korrigiertem Test stimmte Q5_1 mit ggml überein (relativer Fehler $1,5 \cdot 10^{-2}$). Seither laufen alle Schichten über MMQ; die Expertenrechnung pro 2048-Token-Block sank von 590 auf 280 ms.

Später kamen weitere Formate hinzu, jeweils weil ein konkretes Modell sie brauchte: Q5_0-Experten für Swift-1.5 Q4_K_L und OrcaRouter Q4_K_S, Q6_K für Unsloths UD-Q6_K_XL, N-Gramm-Tabellen in Q8_0 (Unsloth, Swift) und Q5_1 (eine Q5_K_M-Fassung). Jedes Format bekam einen Paritätstest gegen ggml.

Fork-Beitrag: Q5_0 genauer als die Vorlage

Q5_0 speichert jedes Gewicht als 5-Bit-Zahl q zwischen 0 und 31 und meint damit den Wert $d \cdot (q - 16)$. Das Skalarprodukt mit den 8-Bit-Aktivierungen a zerfällt in einen Hauptterm $d \sum q a$ und einen Korrekturterm $-16 d \sum a$. Die Vorlage aus llama.cpp nimmt für $\sum a$ eine vorab gespeicherte, auf 16 Bit gerundete Summe der *ungerundeten* Aktivierungen. Der Fork bildet die Summe aus genau den quantisierten Werten, mit denen auch der Hauptterm rechnet – die Konvention von ggmls CPU-Backend, der Upstream für Q5_1 bereits folgte. Der Fehler im synthetischen Paritätstest sank von $2,73 \cdot 10^{-2}$ auf $1,24 \cdot 10^{-2}$, gleichauf mit Q5_1 und Q8_0; die Entpackung stimmt mit ggml in jedem Bit überein.

Für den Installer kam schließlich eine kleine, aber nützliche Funktion hinzu: `strata --expert-support` beantwortet ohne Modell und ohne GPU, ob die Engine für eine bestimmte Kombination aus gate/up- und down-Format und Matrixgröße GPU-Kerne besitzt. So kann ein Installationsskript eine heruntergeladene Datei prüfen, bevor es stundenlang packt.

Linux-Speicher richtig ausnutzen

Viele Entscheidungen in Upstream sind von Windows geprägt, wo die meisten Nutzer sitzen. Eine davon hatte unter Linux drastische Folgen. Windows' Treibermodell WDDM verweigert Speicherzuteilungen, wenn zu viel Arbeitsspeicher für die GPU gepinnt ist; Upstream begrenzte das Pinnen deshalb auf 8 GiB der Arena. Diese Grenze galt aber auch unter Linux, wo es sie nicht braucht. Bei den kleinen GSQ-RCO-Dateien fiel das kaum auf; bei UD-Q4_K_XL mit 72 GiB Experten blieb fast die gesamte Arena ungepinnt. Jede Kopie zur GPU lief dann über einen Umweg: CPU kopiert in einen gepinnten Zwischenpuffer, erst dann übernimmt die DMA-Engine.

Fork-Beitrag: Ganze Arena pinnen

Der Fork pinnt unter Linux die gesamte Arena und dimensioniert den Ring der Kopierpuffer nach der tatsächlichen Größe der Experten (384 Plätze für kleine i-Quants, etwa 128 für Q4-Experten bei 512 MiB Budget), damit der größere Ring nicht den VRAM sprengt. Prompt-Verarbeitung mit UD-Q4_K_XL: von 400 auf rund 890 Token/s, Decode von 72 auf 76 Token/s. Upstream hat dieselbe Konsequenz kurz darauf in Version 0.1.31 gezogen.

Ein zweiter Punkt betrifft die Seitengröße. Normaler Speicher wird in Seiten zu 4 KByte verwaltet. Eine Arena von 78 GByte besteht damit aus fast 20 Millionen Seiten, deren Adressübersetzungen der Prozessor in einem kleinen Puffer, dem TLB, zwischenspeichert. Streamt die CPU ganze Experten von 3 MByte, verfehlt sie diesen Puffer ständig.

FACHBEGRIFF: Huge Pages und TLB

Der *Translation Lookaside Buffer* (TLB) speichert die Übersetzung virtueller in physische Adressen für die zuletzt benutzten Speicherseiten. Er fasst nur einige tausend Einträge. Mit *Huge Pages* von 2 MByte statt 4 KByte deckt jeder Eintrag 512-mal mehr Speicher ab. Linux bietet sie als *Transparent Huge Pages* (THP) an: Ein Programm markiert einen Bereich mit `madvise(MADV_HUGEPAGE)`, der Kernel erledigt den Rest.

Fork-Beitrag: Transparent Huge Pages

Der Code versprach in einem Kommentar die „beste Ausrichtung, die das Betriebssystem kostenlos hergibt“, legte die Arena unter Linux ohne vorkonfigurierten Hupage-Pool aber mit 4-KByte-Seiten an. Der Fork richtet sie auf 2 MByte aus und markiert sie für THP; 75 der 78 GByte landen auf großen Seiten. Die CPU-Experten pro Verify-Fenster brauchen 11,0 statt 12,5 ms, das Laden der Experten beim Start 37 statt 62 Sekunden.

Der dritte Speicherbefund betraf die N-Gramm-Tabelle. Bei gewöhnlichen GGUF-Dateien liegt sie nicht als IQ4_NL mit 28,8 GByte vor, sondern als Q8_0 mit rund 54 GByte. Upstreams schneller Direktleser kann nur IQ4_NL; Q8_0 lief daher über `mmap`, das jede Zeile einzeln von der Platte nachlädt. Auf dem ZFS-Dateisystem des Servers verbrachte ein Prompt so 4 bis 20 Sekunden mit Tabellenzugriffen, und jeder neue Text berührt neue Zeilen.

Fork-Beitrag: Die Tabelle im RAM

Mit `--ple-io ram` liest die Engine die gesamte Tabelle beim Start mit parallelen Lesern in einen anonymen, mit Huge Pages hinterlegten Speicherbereich und verwirft die Dateizuordnung. Auf einem Server mit 450 GByte RAM ist das kein Problem. Die Zeit für Tabellenzugriffe über einen ganzen Benchmark sank von 41 Sekunden auf 0,12 Sekunden; die Prompt-Geschwindigkeit bei 32 000 Token stieg von 1420 auf 2970 Token/s. Ein späterer Commit lässt 16 Threads die Seiten vorab anfassen, bevor sie gesperrt werden: 2 GiB/s statt 0,47 GiB/s beim Kaltstart. Upstream kennt `--ple-io ram` inzwischen ebenfalls.

Prompts auf zwei Karten

Auf einer einzelnen GPU leiht sich der Prompt-Pfad seine Puffer aus dem Experten-Cache und gibt sie danach zurück. Bei einer Schichtaufteilung auf mehrere Karten hatte Upstream 0.1.28 das abgeschaltet: Jede Stufe reservierte eigene Puffer für die gesamte Sitzung, der automatische Modus fiel auf feste 2048-Token-Blöcke zurück, und jeder Block streamte alle nicht im Cache liegenden Experten erneut.

Fork-Beitrag: Leihen auf jeder Stufe

Der Fork gibt jeder Stufe einen eigenen „Verleiher“, der die letzten Plätze ihres Caches ausleiht. Die Blockgröße ist die größte, die jede Stufe verleihen kann; beim Verleihen und Wiederauffüllen werden nur die Einträge der eigenen Schichten gelöscht und wiederhergestellt. Weil die Puffer nicht mehr dauerhaft reserviert sind, wachsen die Caches von 7866 auf 8884 Plätze, die Blöcke von 2048 auf 8192 Token. Ergebnis: Prompts mit 32 000 Token 2930 statt 1410 Token/s, Decode 104 statt 96,6 Token/s. Ein Nadel-im-Heuhaufen-Test bei 32K, 128K und 258K bestätigte, dass die Engine weiterhin jede Stelle des Kontexts findet. Upstream hat in 0.1.30 und 0.1.32 eine eigene Form dieser „Prompt-Leihen“ eingeführt, der der Fork heute folgt.

Eine zweite Beobachtung betraf kurze Prompts. Passt ein Prompt in einen einzigen Block, arbeiten die Stufen nacheinander: Erst liest Karte 0 ihre Schichten, dann Karte 1. Die jeweils andere Karte sitzt untätig daneben – obwohl sie eine eigene PCIe-Anbindung und eigene Rechenwerke hat.

Fork-Beitrag: Die untätige Karte hilft

Während eine Stufe einen kurzen Prompt liest, streamt die GPU der untätigen Stufe einen Teil der nicht im Cache liegenden Experten dieser Schicht über ihre eigene PCIe-Verbindung, rechnet sie und liefert für jedes Token die gewichtete Summe zurück. Das verdoppelt effektiv die PCIe-Bandbreite für den Prompt. Prompts von 2000 Token werden 28 bis 34 Prozent schneller (z. B. 981 auf 1310 Token/s); ab 4000 Token, wo sich die Stufen ohnehin überlappen, bleibt alles gleich. Abschaltbar mit `STRATA_PREFILL_HELP=0`.

Dabei trat ein subtiles Problem auf, das zeigt, wie ernst das Projekt Reproduzierbarkeit nimmt. Die erste Version addierte die Beiträge der Helfer-GPU mit atomaren Operationen auf. Das ist schnell, aber die Reihenfolge der Additionen hängt davon ab, welcher Thread zuerst fertig wird.

FACHBEGRIFF: Nicht-assoziative Gleitkommazahlen

Für Gleitkommazahlen gilt $(a+b)+c \neq a+(b+c)$ im Allgemeinen, weil nach jeder Addition gerundet wird. Addieren viele GPU-Threads per `atomicAdd` in eine Speicherzelle, ändert sich die Reihenfolge von Lauf zu Lauf – und damit die letzten Bits des Ergebnisses. Bei einem Sprachmodell kann das an knappen Stellen ein anderes Token und damit einen anderen Text ergeben.

Der Fork ersetzte die atomaren Additionen durch eine Summierung, die für jedes Token die Beiträge in fester Reihenfolge addiert. Seither ist ein Prompt mit Hilfe der zweiten Karte Bit für Bit reproduzierbar, was der Paritätstest der Gesprächsparkplätze (siehe unten) explizit prüft.

Ein Server-Prozessor rechnet anders

Der Parameter `--pcie-frac` bestimmt, welcher Anteil der Cache-Fehlgriffe über PCIe zur GPU geht statt von der CPU gerechnet zu werden. Auf einem Sechskern-Desktop ist ein großer Anteil sinnvoll. Der EPYC des Servers hat dagegen 24 Kerne und acht Speicherkanäle mit rund 105 GByte/s – doppelt so viel wie ein Desktop-PC. Hier ist die CPU schneller fertig, als PCIe die Daten liefert, und jede DMA-Kopie konkurriert zudem mit dem Prompt-Pfad um den Bus. Der Fork betreibt den Server daher mit `--pcie-frac 0`: Alle Fehlgriffe rechnet die CPU. Der neue Installer schlägt das automatisch vor, sobald er mindestens 32 Threads findet.

Folgefragen, die nicht neu lesen

Für Agenten zählt vor allem eines: Wie schnell beginnt die Antwort auf die nächste Runde? Der Agent schickt den gesamten Verlauf, die Engine erkennt im Idealfall, dass sie alles bis auf die letzten paar hundert Token schon kennt, und liest nur den Rest. Beim Messen fiel auf, dass das nur in 9 bis 10 von 14 Fällen klappte. In den übrigen fiel die Engine auf den Checkpoint am Rundenbeginn zurück und las die gesamte letzte Antwort samt neuer Nachricht erneut – 0,7 bis 0,9 statt 0,3 Sekunden bei 8000 Token Kontext. Die Analyse förderte zwei unabhängige Ursachen zutage.

Fork-Beitrag: Nur festschreiben, was ausgeliefert wurde

Ein Verify-Fenster schrieb alle akzeptierten Positionen in die Sitzung – auch dann, wenn die Antwort mitten im Fenster endete, etwa durch ein Ende-Token oder das Erreichen von `max_tokens`. Die Sitzung enthielt dann Entwurfs-Token, die der Client nie gesehen hatte, und passte nicht mehr zum nächsten Prompt. Der Fork zählt zuerst, welche Token das Fenster tatsächlich ausgibt, und schreibt genau diese fest. Ergebnis: 14 von 14 Folgefragen setzen auf der laufenden Sitzung auf, in 280 bis 316 ms.

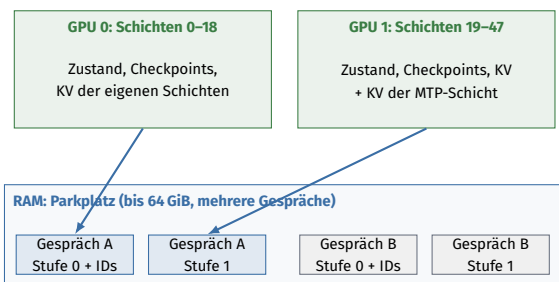


Abbildung 5. Parken mit Schichtaufteilung: Jede Stufe sichert ihren eigenen Teil und lädt ihn auf ihre eigene GPU zurück. Die Token-IDs, nach denen gesucht wird, trägt das Abbild der ersten Stufe.

Die zweite Ursache ist eine Eigenheit von Tokenizern, die selbst erfahrene Entwickler überrascht.

FACHBEGRIFF: BPE und mehrdeutige Zerlegung

Tokenizer nach dem Verfahren *Byte Pair Encoding* (BPE) zerlegen einen Text auf eine bestimmte, kanonische Weise. Ein Modell erzeugt aber Token für Token und kann denselben Text auch anders „buchstabieren“ – etwa „Daten“ + „bank“ statt „Datenbank“. Wird die Antwort später als Text zurückgeschickt und neu zerlegt, entsteht an solchen Stellen eine andere Token-Folge, obwohl der Text identisch ist.

Fork-Beitrag: Die eigenen Token-IDs behalten

Weil die rekurrenten Schichten nur Token für Token übereinstimmen dürfen, genügt eine einzige abweichende ID, und die Engine verwirft den Rest. In einem diagnostizierten Fall stimmte der Text auf 8098 Token überein, die Engine konnte aber nur 7963 wiederverwenden. Der Server merkt sich nun für die letzten acht abgeschlossenen Anfragen die tatsächlichen Token-IDs (Prompt plus Antwort) und deren Text. Beginnt ein neuer Prompt mit genau diesem Text, übernimmt er die gespeicherten IDs und zerlegt nur den Rest neu. Bilder, abgebrochene oder fehlgeschlagene Anfragen sind ausgenommen.

Gespräche parken – auch auf zwei Karten

Upstream kann seit Ende September mehrere Gespräche im RAM *parken*: Ist eine Anfrage fertig, wandert der komplette Zustand – laufender rekurrenter Zustand, Checkpoints, belegte KV-Seiten, KV-Cache der MTP-Schicht – in einen RAM-Puffer. Kommt der nächste Beitrag dieses Gesprächs, wird er zurückgeladen, statt den Kontext neu zu lesen. Für Agenten, die sich mit einem Web-Chat abwechseln, ist das Gold wert. Nur: Mit Schichtaufteilung verweigerte Upstream die Funktion schon beim Start. Auf dem Server, der beide Karten braucht, war sie damit nutzlos.

Fork-Beitrag: Parken pro Stufe

Im Fork legt jede Stufe ein eigenes Abbild an (Abbildung 5): mit ihrem eigenen laufenden Zustand, ihren Checkpoint-Zuständen und dem KV-Cache ihrer Schichten; der KV-Cache der MTP-Schicht reist mit der letzten Stufe. Das Abbild der ersten Stufe trägt die Token-IDs, nach denen der Cache sucht, und nimmt die übrigen Teile als „stage parts“ mit. Die Checkpoint-Kette wird zum Sichern in Stufenansichten zerlegt und beim Wiederherstellen zusammengesetzt. Vor dem Zurückladen wird *jeder* Teil validiert, bevor auch nur einer angewendet wird – ein halb wiederhergestelltes Gespräch wäre schlimmer als gar keines. Ein Gespräch mit knapp 2000 Token wird in rund 150 ms geparkt und in 25 ms zurückgeladen. Zwei Gespräche zu je 8000 Token im Wechsel: Die erste Antwort nach einem Wechsel beginnt nach 0,63 statt 4,6 Sekunden. Geprüft mit Upstreams eigenem Paritätstest: Der Zustand des Hauptmodells nach dem Wechsel A-B-A ist Byte für Byte identisch, auch bei aktiver Helfer-GPU und unter Speicherdruck.

Eine Einschränkung bleibt: Upstream kann auf einer ein-

Ein Verify-Fenster, zwei Sequenzen

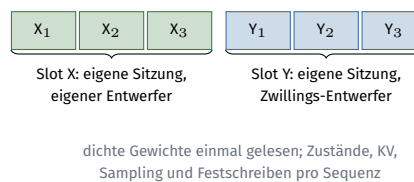


Abbildung 6. Batch-2: Das Fenster wird in zwei Segmente geteilt, jedes mit eigenem rekurrentem Zustand, eigenem KV-Cache und eigenen Sampling-Parametern.

zelnen GPU beim nächsten Parken bereits gesicherte KV-Bereiche wiederverwenden und nur den Zuwachs kopieren. Der Fork kopiert bei aufgeteilten Modellen jedes Mal alles. Bei 150 ms pro Vorgang ist das verkraftbar.

Batch-2: zwei Gespräche in einem Fenster

Die ehrgeizigste Arbeit des Forks ist zugleich die jüngste. Upstream-Strata bearbeitet genau eine Anfrage zur Zeit; eine zweite wartet in der Schlange. Für einen einzelnen Nutzer ist das richtig. Arbeiten aber zwei Agenten gleichzeitig, liegt die Frage nahe, ob man nicht wie vLLM beide gemeinsam durch das Modell schicken kann. Die Idee: Das Verify-Fenster nimmt Token beider Gespräche auf, die dichten Gewichte werden einmal für beide gelesen, und die CPU rechnet die Experten des einen Gesprächs, während die GPU den Mixer des anderen rechnet.

Der Ansatzpunkt existierte schon: Upstream teilt ein Fenster intern in zwei Token-Gruppen, ursprünglich für den (verworfenen) Versuch, eine Sequenz in zwei Hälften zu überlappen. Batch-2 macht aus Gruppe A die eine und aus Gruppe B die andere Sequenz. Der Aufwand steckt im Detail: Jeder Zustand, der an eine Sequenz gebunden ist, muss verdoppelt werden – die DeltaNet-Rekurrenz und ihr Faltungsverlauf, die KV-Pools und Indexer-Daten der Sparse Attention, der Verlauf für die N-Gramm-Tabelle, Positionszähler, die Momentaufnahmen für das Festschreiben. Bei Schichtaufteilung braucht jede Stufe eine zweite Sitzung.

Die Arbeit ist in Meilensteine gegliedert und jeweils durch Paritätstests abgesichert:

M1: Segmentiertes Fenster

Zwei Sequenzen, erst einzeln, dann gemeinsam in Fenstern der Form 1+1, 2+2, 3+2 und 1+4 über 300 Schritte: gleiche Token-Wahl, jede Zeile der Ausgabewahrscheinlichkeiten *bitgleich*.

M2: Gemeinsames Dekodieren

Ein „Zwilling-Entwerfer“ teilt Gewichte und Entwurfskopf mit dem ersten, hat aber eigenen KV-Ring, Puffer und CUDA-Stream (rund 19 MiB). Beide Sequenzen dekodieren im Gleichschritt mit MTP-Entwürfen; 400 Token je Sequenz, identische Ausgaben bei jeder Fensterbreite.

M3: Server

Pro Sequenz eigene Sampling-Parameter (Temperatur, Top-p/k, Seed, Wiederholungsstrafe). Ein Protokoll mit Anfrage-IDs zwischen Python-Server und Engine (GEN ... id=N, T@N, DONE@N, STOP N). Kommt eine zweite Anfrage, während eine

erste dekodiert, wandert die laufende nach dem aktuellen Fenster in Slot Y (154 ms), die neue wird in Slot X gelesen, dann laufen beide im Gleichschritt. 138 Tests im Server.

Die Messungen fielen ernüchternder aus als die Schätzung von 1,3 bis 1,5. Zwei Sequenzen gemeinsam bringen nur das 1,12- bis 1,20-Fache des Einzelbetriebs. Der Grund steht im Projekt-Dokument selbst: Die Kosten eines Fensters wachsen fast linear mit seinen Zeilen, rund 10 bis 11 ms pro Zeile auf dieser Maschine, weil jede Zeile ihre eigenen Experten wählt und zwei verschiedene Gespräche kaum Experten gemeinsam haben. Der Anteil, den ein Batch teilt – die dichten Gewichte –, ist bei diesem Modell klein. Genau hier zeigt sich der grundlegende Unterschied zu vLLM: Bei einem Modell, das ganz im VRAM liegt, ist Batching fast gratis; bei einem, dessen Experten größtenteils auf der CPU gerechnet werden, kaum. In der Praxis erreichen vier parallele Anfragen 97 bis 99 Token/s in Summe statt 82 seriell, für 1,1 GiB zusätzlichen VRAM.

Fork-Beitrag: Graphen vorab aufzeichnen

Jede Kombination der Fensterbreiten von X und Y braucht einen eigenen CUDA Graph. Zunächst zeichnete die Engine sie bei Bedarf auf – unter Last fehlte dann der Speicher dafür, weil die Experten-Caches den VRAM gefüllt hatten. Jetzt werden alle Formen beim Start aufgezeichnet, in 320 MiB pro Karte, die der Cache von vornherein freilässt.

Batch-2 ist im Fork optional (`--batch2-cells`) und standardmäßig aus; der Installer schlägt es nur vor, wenn zwei Karten mit mindestens 20 GByte vorhanden sind. Die abschließende Messung mit echten, parallel arbeitenden Agenten (M4) steht noch aus.

Ein Installer, der begründet

Den Abschluss bildet ein Installer für ein nacktes Debian 12 oder 13. `install.sh` richtet Systempakete, den NVIDIA-Treiber (mit einem Neustart) und CUDA 13.3 aus NVDIAs Paketquelle ein, baut eine Python-Umgebung und kompiliert die Engine für die vorhandenen Karten. Mit `--gguf` prüft er eine Modelldatei: Architektur, die Expertenformate jeder Schicht gegen die GPU-Kerne der Engine (per `--expert-support`), den RAM für die Arena. Danach schlägt er Einstellungen für genau diese Hardware vor und begründet jede einzelne – ein Auszug aus der Logik:

- **Kontext:** der längste aus 262K, 128K, 64K und 32K, bei dem der int8-KV-Cache (13 Schichten $\times$ 1056 Byte pro Token) höchstens ein Viertel des freien VRAM jeder Karte belegt.
- **N-Gramm-Tabelle im RAM**, wenn neben den Experten und 24 GiB Reserve noch Platz ist.
- **--pcie-frac 0** ab 32 CPU-Threads.
- **Parkplatz:** die Hälfte des verbleibenden RAM, höchstens 64 GiB, mit 4 bis 20 Plätzen.
- **Batch-2** nur mit zwei Karten ab 20 GByte.

Nach Bestätigung baut er Pack und MTP-Kopf, schreibt Konfiguration und Startskript und richtet auf Wunsch einen systemd-Dienst ein. Upstreams eigener Ein-Klick-Installer bleibt für die GSQ-RCO-Modelle erhalten.

Bilanz: Was der Fork beigetragen hat

Tabelle 3 fasst die Beiträge des Forks zusammen. Auffällig ist ihr Charakter: Kaum einer ändert die Grundarchitektur von Strata. Die Aufteilung auf drei Speicherebenen, die gleichzeitige Arbeit von CPU, GPU und PCIe, der adaptive Cache, das exakte spekulative Dekodieren – all das stammt aus Upstream und trägt auch im Fork die Last. Der Fork hat diese Architektur auf eine Umgebung übertragen, für die sie nicht gebaut war, und dabei die Stellen gefunden, an denen Annahmen der Basis nicht mehr stimmten: dass Speicher wie unter Windows gepinnt werden muss, dass die Tabelle auf die SSD gehört, dass die CPU schwach ist, dass es nur eine GPU gibt, dass nur ein Gespräch zählt.

Gleichzeitig hat der Fork die Arbeitsweise der Basis übernommen, und das ist vielleicht sein wichtigster Beitrag zur Glaubwürdigkeit. Jede Änderung kommt mit Messwerten vorher und nachher, jeder neue Rechenkern mit einem Paritätstest gegen ggml, jede Änderung an Zuständen mit dem Nachweis, dass der Zustand Byte für Byte derselbe bleibt. Wo eine Optimierung das Ergebnis verändern würde – etwa die atomaren Additionen der Helfer-GPU –, wird sie umgebaut, bis sie es nicht mehr tut. Und wo eine Idee weniger bringt als erhofft, wie Batch-2, dokumentiert das Projekt das offen, samt Erklärung.

Grenzen

Strata bleibt eine Engine für eine einzige Modellfamilie. Wer morgen ein anderes Modell ausprobieren will, ist mit `llama.cpp` besser bedient, und wer hundert Nutzer gleichzeitig bedienen muss und die passenden GPUs hat, mit vLLM. Der Fork verengt den Einsatzbereich noch weiter: Er ist auf Linux, NVIDIA-Karten und eine Schichtaufteilung über zwei GPUs ausgelegt. Eine einzelne GPU in einem LXC-Container kann scheitern, wenn der Experten-Cache den VRAM füllt; Parken mit Schichtaufteilung kopiert jedes Mal den vollen Zustand; der Einzelkarten-Prüfmodus der Übergabe verweigert das Parken. Batch-2 ist experimentell, seine abschließende Messung mit realen Agenten steht aus.

Auch die Vergleichszahlen verdienen eine Einordnung. Die Werte gegenüber dem Stand „vorher“ mischen Fortschritte von Upstream (0.1.28 bis 0.1.32) mit denen des Forks; die einzelnen Commits weisen ihre Wirkung zwar jeweils isoliert aus, addieren sich aber nicht einfach. Der Vergleich mit FreeToken beruht auf einer anderen Quantisierung (NVFP4) und einer anderen Parallelisierung; er zeigt Größenordnungen, keinen Laborvergleich. Und das Decode-Tempo enthält die akzeptierten Entwürfe des spekulativen Dekodierens, hängt also vom Text ab: Code, der Bekanntes wiederholt, ist schneller als frei formulierte Prosa.

Fazit

Strata zeigt, wie viel Leistung in gewöhnlicher Hardware steckt, wenn man eine Engine um ein konkretes Modell herum baut, statt ein Modell in eine allgemeine Engine zu zwingen. Die Zahlen sind beeindruckend: Ein Modell mit 125 Milliarden Parametern schreibt auf einer 12-GByte-Karte schneller, als man lesen kann. Möglich wird das durch ein Modell, das

Tabelle 3. Die Beiträge von arcfork-strata im Überblick (Messungen auf 2× RTX 3090, UD-Q4_K_XL bzw. Swift-1.5, 262K Kontext)

Beitrag	Problem in der Basis	Wirkung	Verhältnis zu Upstream
GPU-Kerne für K-Quants, Q5_1, Q8_0, Q5_0, Q6_K	gewöhnliche GGUFs nicht auf der GPU rechenbar	UD-Q4_K_XL, Swift, Orca, UD-Q6_K_XL lauffähig; Q5_0 genauer als Vorlage	Grundformate heute auch upstream; Q5_0, Q6_K fork-eigen
Q5_1 über MMQ	Fehler im Paritätstest	Expertenrechnung pro Block 590 → 280 ms	–
Ganze Arena pinnen	8-GiB-Grenze aus Windows galt auch unter Linux	Prompt 400 → 890 Tok/s	upstream ab 0.1.31
Transparent Huge Pages	Arena in 4-KB-Seiten	CPU-Experten 12,5 → 11,0 ms; Start 62 → 37 s	fork-eigen
N-Gramm-Tabelle im RAM	Q8_0-Tabelle zeilenweise per mmap	Tabellenzeit 41 s → 0,12 s; 32K-Prompt 1420 → 2970 Tok/s	upstream in eigener Form
Prompt-Leihen auf jeder Stufe	bei Schichtaufteilung feste Puffer, kleine Blöcke	32K-Prompt 1410 → 2930 Tok/s, Decode +8 %	upstream in eigener Form (0.1.30/0.1.32)
Helfer-GPU für kurze Prompts	untätige Stufe bei Ein-Block-Prompts	2K-Prompts +28–34 %, deterministisch	fork-eigen
Nur Ausgegebenes festschreiben	Sitzung enthielt ungesehene Entwürfe	14/14 statt 9–10/14 Folgefragen ohne Neulesen	fork-eigen
Eigene Token-IDs behalten	BPE-Mehrdeutigkeit beim Neuzerlegen	Folgefragen setzen auf Sitzung auf	fork-eigen
Parken mit Schichtaufteilung	bei zwei GPUs verweigert	Gesprächswechsel 4,6 → 0,63 s	fork-eigen
Batch-2	nur eine Anfrage zur Zeit	1,12–1,2× für zwei, 4 parallel 97–99 statt 82 Tok/s	fork-eigen, experimentell
Installer mit Begründungen	Ein-Klick-Setup nur für GSQ-RCO	Debian-Server in einem Schritt	fork-eigen

pro Token nur zwei Prozent seiner Experten braucht, und eine Engine, die jedes Byte dorthin legt, wo es am billigsten zu lesen ist, und jedes Bauteil gleichzeitig beschäftigt.

arcfork-strata zeigt die zweite Hälfte der Geschichte: wie eine solche Spezialengine in einer anderen Umgebung wächst. Auf einem Server mit zwei RTX 3090, viel RAM und vielen Kernen werden plötzlich andere Dinge wichtig – die Seitengröße des Speichers, die Tokenisierung zurückgeschickter Antworten, die Frage, wer gerade untätig ist. Der Fork hat diese Stellen gefunden und mit überschaubarem Code – rund 3600 Zeilen in 38 Dateien – die Prompt-Verarbeitung mehr als verdoppelt, das Schreibtempo um 60 Prozent gesteigert und den Wechsel zwischen Agenten-Sitzungen von Sekunden auf Sekundenbruchteile verkürzt. Für Programmier-Agenten, die in jeder Runde zehntausende Token Kontext mitschicken, ist das der Unterschied zwischen einem Werkzeug, auf das man wartet, und einem, mit dem man arbeitet.

Quellen und Links

Strata (Upstream): github.com/Niko1221/Strata – Quelltext, Dokumentation (docs/DETAILS.md, docs/MULTI_GPU.md, docs/UNSLOTH_Q4.md) und das technische Paper docs/paper/Strata-Paper.pdf

arcfork-strata: interner Fork auf Basis von Upstream 0.1.32; Fork-README, Batch-2-Dokument docs/BATCH2.md, Installer [install.sh](https://tools/install.sh) und tools/install/; die Commit-Nachrichten enthalten zu jeder Änderung die Messwerte

Modell: Qwen3.8-Flash-Next (huggingface.co/Qwen); Quantisierungen von ISTA-DASLab (GSQ-RCO), Unsloth (UD-Q4_K_XL), Feintuning Swift 1.5 von UkisAI

Andere Engines: llama.cpp (github.com/ggml-org/llama.cpp), vLLM (github.com/vllm-project/vllm), FreeToken (github.com/FlashML-org/FreeToken)

Hintergrund: Leviathan et al., „Fast Inference from Transformers via Speculative Decoding“ (ICML 2023); Kwon et al., „PagedAttention“ (SOSP 2023); Yang et al., „Gated Delta Networks“ (2024); KTransformers (SOSP 2025); Fiddler, MoE-Infinity (2024)